Ajuste fino y selección de modelos

Introducción al Aprendizaje Automático

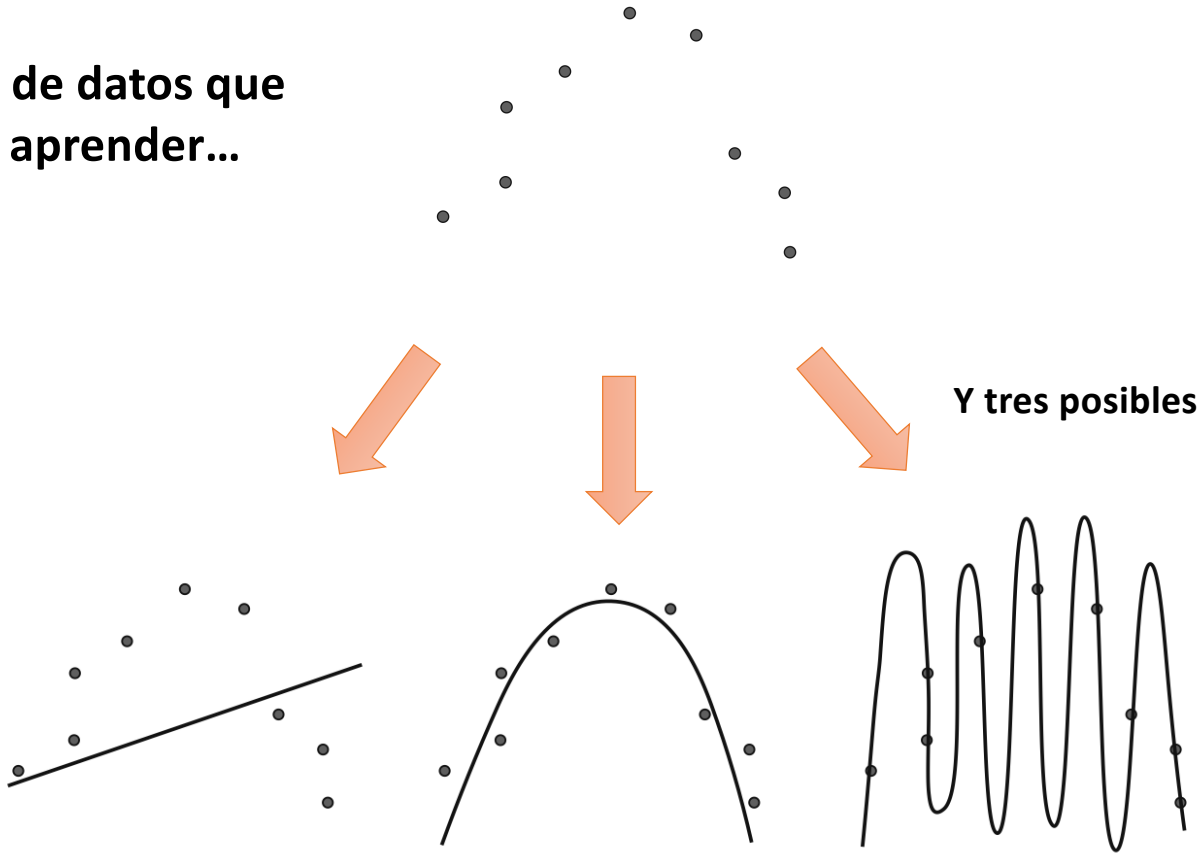
Sebastián Ventura Soto

Infraajuste / Sobreajuste
(Underfitting / Overfitting)

- Cuando se pone un modelo en producción, puede que los modelos obtenidos no sean los esperados.
- Problemas que se pueden presentar:
  - **Infraajuste (*underfitting*)**: El modelo no aprende bien la relación existente entre las características y la variable a predecir.
  - **Sobreajuste (overfitting)**: El modelo ha memorizado la relación existente entre las características y la variable, sólo para los casos usados para entrenar el modelo.

Infraajuste / Sobreajuste
(Underfitting / Overfitting)

Un conjunto de datos que queremos aprender…



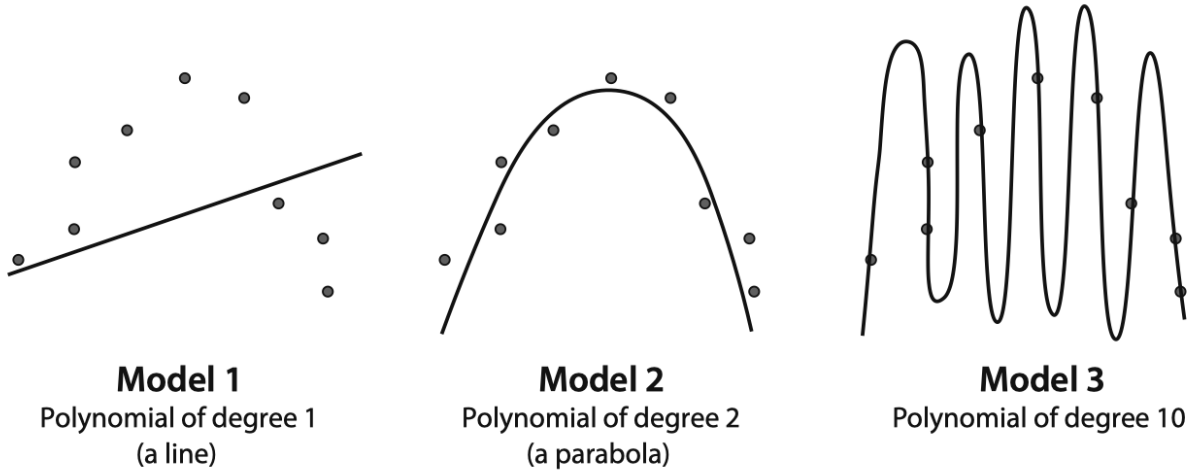
Y tres posibles modelos

Model 1
Polynomial of degree 1
(a line)

Model 2
Polynomial of degree 2
(a parabola)

Model 3
Polynomial of degree 10

Infraajuste / Sobreajuste
(Underfitting / Overfitting)

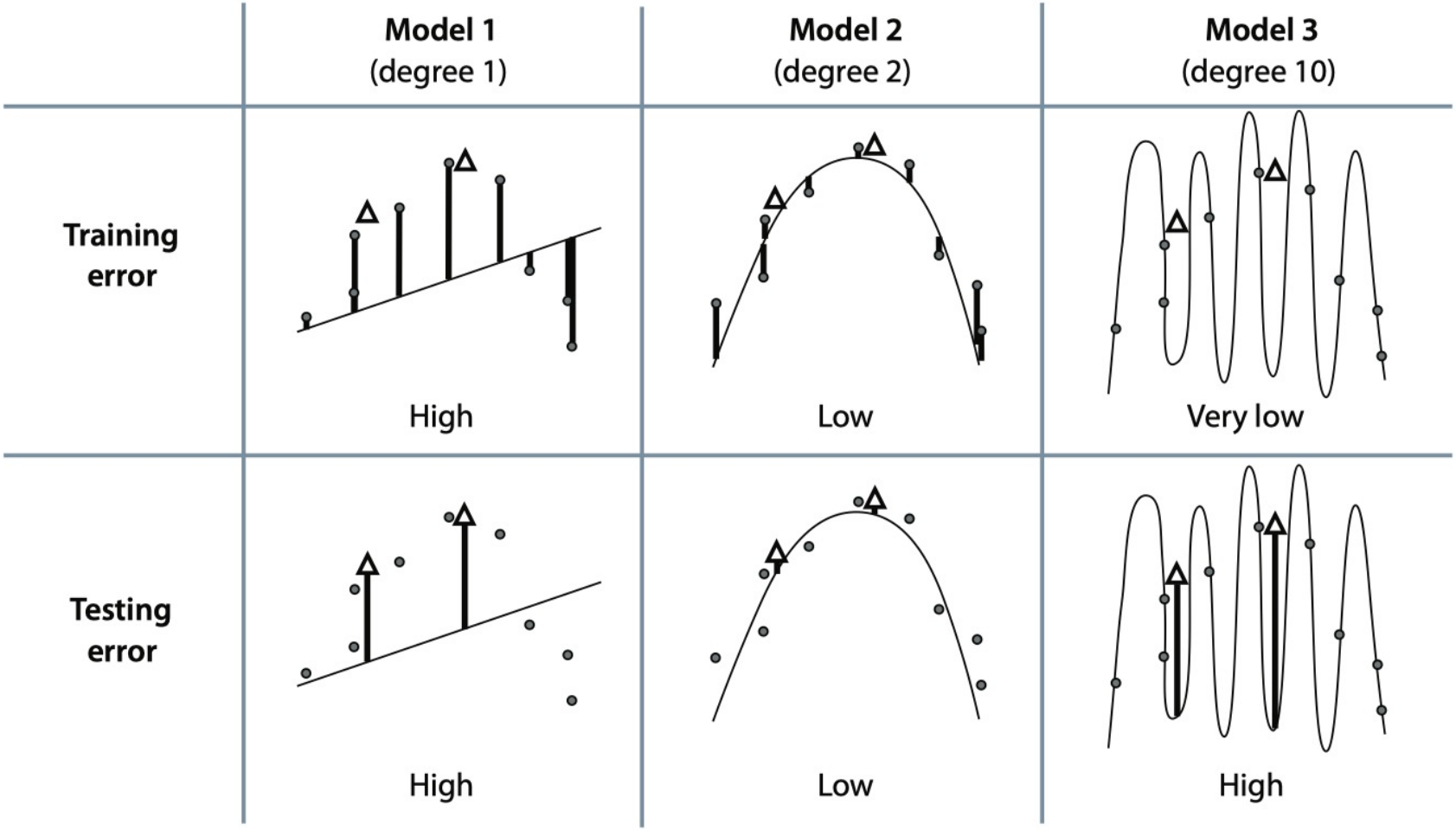


- Nuestro sentido común nos dice que el mejor modelo es el segundo (refleja más fielmente la tendencia de los datos).
- Los modelos demasiado simples suelen estar infraentrenados
- Los modelos demasiado complejos suelen estar sobreentrenados.
- ¿Cómo podemos medir sobreajuste o el infraajuste?

Evaluación (test) de modelos

- La forma más sencilla de saber que un modelo está sobreajustado es evaluar el modelo.
- Seleccionamos un conjunto de puntos pequeño que no se usará para entrenar el modelo, sino que sólo servirá para evaluarlo.
- Los datos disponibles se dividirán, pues en dos subconjuntos:
  - **Conjunto de entrenamiento**: mayor porcentaje de datos
  - **Conjunto de evaluación o test**: menor porcentaje de datos

Evaluación (test) de modelos



Evaluación (test) de modelos

	Infraajuste	Buen ajuste	Sobreajuste
Training	👎🏼	👍🏼	👍🏼
Test	👎🏼	👍🏼	👎🏼

- Cuál es la mejor distribución de datos entrenamiento:test?
  - Reservar un 20% para test
  - Reservar un 10% para test
- Importante: Los datos de test **nunca** se usan para entrenar un modelo

Validación

- La división de los datos en *train:test* se aplica para decidir cuál es el mejor modelo a nivel de parámetros.
- ¿Cómo proceder cuando hemos de elegir el mejor conjunto de hiperparámetros para un modelo?
- Dividimos los datos en tres subconjuntos:
  - Conjunto de entrenamiento *(train):* Entrenar los modelos
  - Conjunto de validación *(validation): Validar los mejores hiperparámetros*
  - Conjunto de evaluación *(test):* Evaluar el rendimiento del modelo
- Los conjuntos de validación / evaluación suelen ser del mismo tamaño
  - 60:20:20
  - 80:10:10

Evaluación Robusta

Dividir los datos en una sola etapa (60:20:20) puede generar estimaciones de error engañosas. Dependemos de la "suerte" al realizar el corte, especialmente con pocos datos.

Evaluación Robusta

Métodos de Remuestreo

- **Remuestreo (*Resampling*):** Extraer repetidamente muestras del conjunto de datos original, ajustando y evaluando un modelo nuevo en cada iteración.

  Objetivo: Obtener una estimación matemática mucho más estable, fiable y realista del rendimiento del modelo al **promediar** los resultados de todas las iteraciones.

- **Técnicas principales:**
  - Validación Cruzada clásica (*K-Fold Cross-Validation*).
  - Validación de Monte Carlo (*Random Sub-sampling*).
  - *Bootstrapping* (Muestreo con reemplazo).

Evaluación Robusta

Validación Cruzada clásica (K-Fold)

El mecanismo k-Fold:

- 1. Se divide el *dataset* en k **particiones de igual tamaño, disjuntas y exhaustivas.**
- 2. Se entrena el modelo k veces. En cada iteración, se usa **1 partición para evaluar** y las **restantes para entrenar**.

- **La Garantía Matemática:** Al terminar el bucle, nos aseguramos de que **todos los patrones del *dataset se han utilizado exactamente una vez para evaluar el modelo.***
- **El resultado final:** El rendimiento real del modelo se calcula como la **media** de los errores de las iteraciones.

Evaluación Robusta

Validación de Montecarlo

En lugar de particiones fijas, en cada iteración se baraja el *dataset* completo y se extrae una muestra aleatoria para Test y el resto para Train.

Ventaja (Flexibilidad): Desvincula el tamaño del test del número de repeticiones. Podemos hacer 1.000 iteraciones manteniendo un test del 20% (en *K-Fold*, un test del 20% obliga a realizar exactamente 5 iteraciones).

Inconveniente (pérdida de cobertura): Al ser particiones aleatorias e independientes, **no son mutuamente disjuntas**. Por pura probabilidad, algunos patrones podrían ser evaluados muchas veces, y otros no ser evaluados nunca.

Uso práctico: Muy útil cuando se requieren simulaciones masivas para calcular con alta precisión la varianza (estabilidad) del modelo ante cambios en los datos.

Automatizando la búsqueda de hiperparámetros

Un modelo real tiene múltiples hiperparámetros (p. ej., tasa de aprendizaje , coeficiente de regularización , tamaño del *mini-batch*). Probar todas sus combinaciones a mano es inviable.

Búsqueda en Cuadrícula (*Grid Search*):

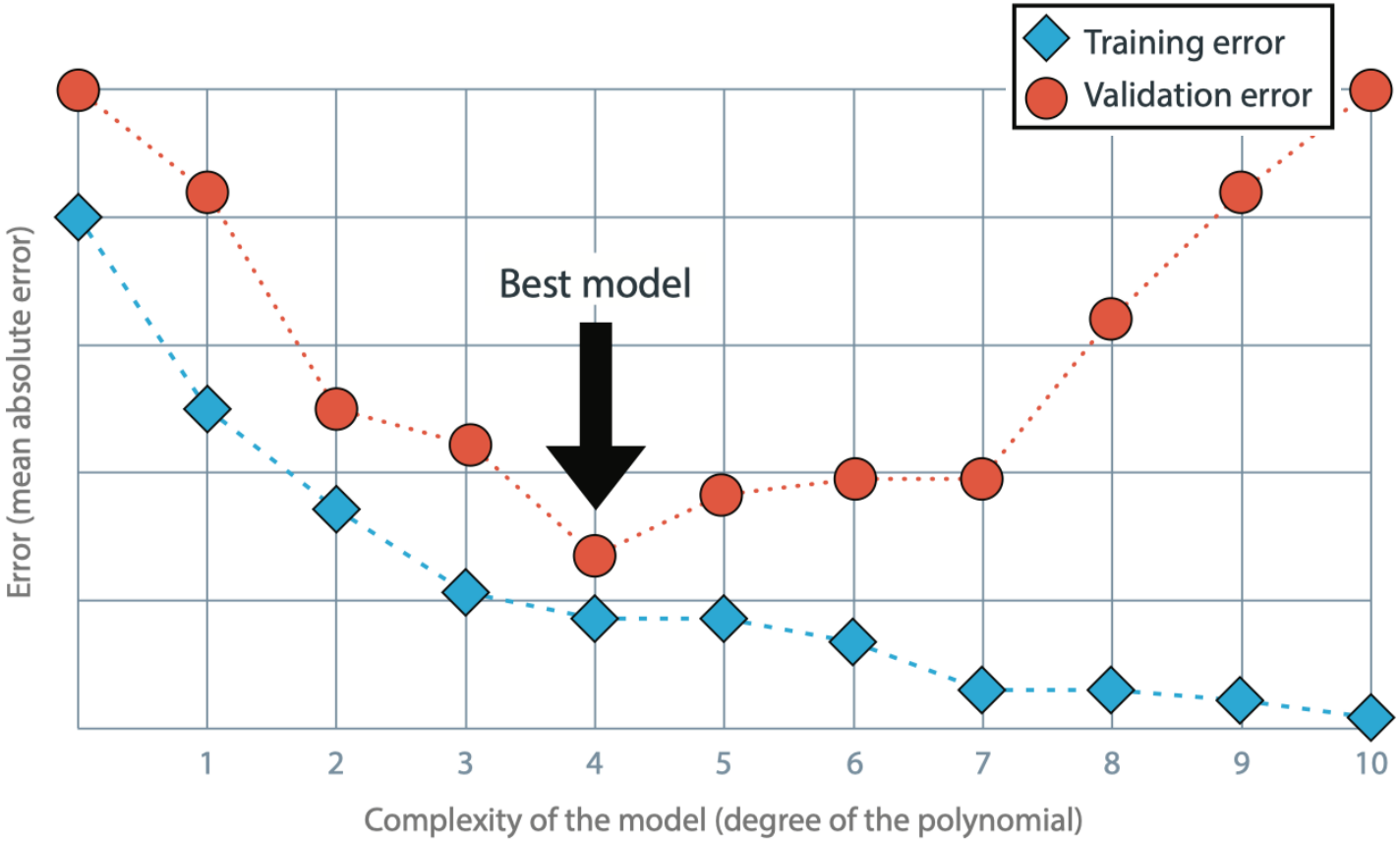
- Fuerza bruta pura. Se define una lista de valores para cada hiperparámetro y el algoritmo evalúa *todas* las combinaciones posibles.
- **Inconveniente:** Sufre de la "maldición de la dimensionalidad". Si añadimos un nuevo hiperparámetro, el tiempo de cómputo se multiplica exponencialmente.

Búsqueda Aleatoria (*Random Search*):

- Se define un rango continuo para cada variable y el algoritmo prueba combinaciones generadas al azar.
- **Ventaja:** Matemáticamente, ha demostrado ser mucho más eficiente que la cuadrícula para encontrar zonas óptimas, especialmente cuando hay hiperparámetros que importan más que otros.

Gráfica de complejidad de modelos

- Sirve para elegir la mejor combinación de hiperparámetros analizando los errores de entrenamiento y validación



- Un beneficio de esta gráfica es que da igual lo grande que sea el dataset o el número de modelos que intentamos probar.

Regularización

- Otra técnica para evitar sobreajuste sin tener que usar conjunto de test
- Se basa en que los modelos simples suelen presentar infraajuste y los complejos sobreajuste
- La ventaja es que no es necesario entrenar varios modelos y comparar. Se entrena una sola vez, pero durante el entrenamiento se intentan ajustar dos objetivos a la vez:
  - Mejora del rendimiento
  - Reducción de la complejidad

Regularización

Un ejemplo sencillo

Problem: Broken roof

Roofer 1
Solution: Bandage
(Underfitting)

Roofer 2
Solution: Shingles
(Correct)

Roofer 3
Solution: Titanium
(Over!tting)

Calidad de la reparación	1000 ml	1 ml	0 ml	
	1 €	100 €	10000 €	Coste de la reparación

Regularización

Un ejemplo más complejo

Modelo genérico de recomendación de películas

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6 + w_7x_7 + w_8x_8 ++ w_9x_9 + w_{10}x_{10} + b,$$

Modelos obtenidos

Model 1: $\hat{y} = 2x_3 + 1.4x_7 - 0.5x_7 + 4$

Model 2: $\hat{y} = 22x_1 - 103x_2 - 14x_3 + 109x_4 - 93x_5 + 203x_6 + 87x_7 - 55x_8 + 378x_9 - 25x_{10} + 8$

¿Cuál parece más interesante?

Regularización

Medidas de complejidad de modelos

- Ambas medidas están basadas en realizar una suma de los valores de los coeficientes del modelo
- Dos tipos de medida (normas)
  - **Norma L1**: Se suma los valores absolutos de los coeficientes del modelo
  - **Normal L2**: Se suma los valores de los cuadrados de los coeficientes del modelo
- El valor del sesgo **no se utiliza** para calcular ninguna de las dos normas

Regularización

Ejemplo de cálculo de normas L1 y L2

Model 1: $\hat{y} = 2x_3 + 1.4x_7 - 0.5x_7 + 8$

Model 2: $\hat{y} = 22x_1 - 103x_2 - 14x_3 + 109x_4 - 93x_5 + 203x_6 + 87x_7 - 55x_8 + 378x_9 - 25x_{10} + 8$

L1 norm:

- **Model 1**: $|2| + |1.4| + |-0.5| = 3.9$
- **Model 2**: $|22| + |-103| + |-14| + |109| + |-93| + |203| + |87| + |-55| + |378| + |-25| = 1{,}089$

L2 norm:

- **Model 1**: $2^2 + 1.4^2 + (-0.5)^2 = 6.21$
- **Model 2**: $22^2 + (-103)^2 + (-14)^2 + 109^2 + (-93)^2 + 203^2 + 87^2 + (-55)^2 + 378^2 + (-25)^2 = 227{,}131$

Regularización

Ejemplo de cálculo de normas L1 y L2

- **Model 1:** $\hat{y} = 2x + 3$
- **Model 2:** $\hat{y} = -x^2 + 6x - 2$
- **Model 3:** $\hat{y} = x^9 + 4x^8 - 9x^7 + 3x^6 - 14x^5 - 2x^4 - 9x^3 + x^2 + 6x + 10$

L1 norm:

- **Model 1:** $|2| = 2$
- **Model 2:** $|-1| + |6| = 7$
- **Model 3:** $|1| + |4| + |-9| + |3| + |-14| + |-2| + |-9| + |1| + |6| = 49$

L2 norm:

- **Model 1:** $2^2 = 2$
- **Model 2:** $(-1)^2 + 6^2 = 37$
- **Model 3:** $1^2 + 4^2 + (-9)^2 + 3^2 + (-14)^2 + (-2)^2 + (-9)^2 + 1^2 + 6^2 = 425$

Regularización

Regresiones lasso y ridge

- Si añadimos un término de regularización a la función de error del modelo, estamos asegurando que, además de minimizar el error, minimizamos su complejidad.
- Hay dos formas de hacer esto:
  - Regresión *lasso*: La función de error incluye la norma L1

    Error lasso = Error de Regresión + Norma L1

  - Regresión *ridge*: La función de error incluye la norma L2

    Error ridge = Error de Regresión + Norma L2

Regularización

Parámetro de regularización

- El parámetro de regularización se utiliza para ponderar en mayor o menor medida el término en las regresiones lasso o ridge:

$$Error = ErrorDeRegresión + \lambda\, TérminoDeRegularización$$

- Suele tomar valores que son potencias de 10: 10, 1, 0.1, 0.01 pero su elección es arbitraria.

Regularización

Gradiente descendente

- Las expresiones del método del gradiente se verán afectadas por el término de regularización.
- Al calcular el término del gradiente, habrá un nuevo término asociado al gradiente de la función de regularización (lasso o ridge)

$$w_j = w_j - \lambda \frac{\partial R}{\partial w_j}$$

Reg. Lasso	$\frac{\partial R}{\partial w_j} = \mathrm{sgn}(w_j)$	$w_j = w_j - \lambda\, \mathrm{sgn}(w_j)$

Reg. Ridge	$\frac{\partial R}{\partial w_j} = 2w_j$	$w_j = w_j - 2\lambda w_j$

Regularización

Efecto de L1 y L2 en los coeficientes del modelo

- Si utilizamos la regularización L1 (regresión lasso), se obtiene un modelo con menos coeficientes.
- Si utilizamos la regularización L2 (regresión ridge), terminamos con un modelo con coeficientes más pequeños.

Si tenemos demasiadas características y queremos deshacernos de la mayoría de ellas, la regularización L1 es la ideal. Si tenemos pocas características y creemos que todas son relevantes, entonces será mejor usar la regularización L2, porque no se deshará de nuestras características útiles.

Regularización

Regresión ElasticNet

- Combina las ideas de las regresiones Lasso y Ridge

Error ElasticNet = Error de Regresión + r Norma L1 + (1-r) Norma L2

- Es un modelo que se usa cuando se dispone de muchos parámetros y que además pueden haber algunos correlados, con lo que interesa:
  - Eliminar algunos parámetros
  - Poner valores pequeños para los parámetros

Regularización en la práctica: Sistemas de Sensores

Regularización L1 (Lasso) Ahorro y Selección:

- **Efecto:** Fuerza a que la mayoría de los coeficientes sean exactamente cero.
- **Uso:** Ideal cuando tenemos **demasiadas características**. El modelo "apaga" los sensores redundantes y nos dice exactamente con cuáles nos basta para predecir el fallo, ahorrando hardware y cómputo.

Regularización L2 (Ridge) Estabilidad y Colinealidad:

- **Efecto:** Reduce el tamaño de los coeficientes, pero sin eliminarlos.
- **Uso:** Ideal cuando tenemos **pocas características y todas son relevantes**. Evita que el modelo dependa en exceso de un solo sensor que podría volverse loco o enviar ruido, distribuyendo el "peso" de la predicción de forma equitativa.

Conclusiones

- Cuando se trata de entrenar modelos, surgen muchos problemas. Dos problemas que surgen con bastante frecuencia son el infraajuste y el sobreajuste.
- El infraajuste se produce cuando utilizamos un modelo muy simple para ajustar nuestro conjunto de datos. El sobreajuste se produce cuando utilizamos un modelo demasiado complejo para ajustarlo a nuestro conjunto de datos.
- Una forma eficaz de diferenciar el sobreajuste y el infraajuste es utilizar un conjunto de datos de prueba.
- Para probar un modelo, dividimos los datos en dos conjuntos: uno de entrenamiento y otro de prueba. El conjunto de entrenamiento se utiliza para entrenar el modelo, y el conjunto de pruebas se utiliza para evaluar el modelo.
- La regla de oro del aprendizaje automático es no utilizar nunca los datos de prueba para entrenar o tomar decisiones en nuestros modelos.
- El conjunto de validación es otra parte de nuestro conjunto de datos que utilizamos para tomar decisiones sobre los hiperparámetros de nuestro modelo.
- Un modelo que no se ajusta lo suficiente tendrá un mal rendimiento en el conjunto de entrenamiento y en el conjunto de validación. Un modelo que se ajuste demasiado tendrá un buen rendimiento en el conjunto de entrenamiento, pero un mal rendimiento en el conjunto de validación. Un buen modelo funcionará bien tanto en el conjunto de entrenamiento como en el de validación.
- El gráfico de complejidad del modelo se utiliza para determinar la complejidad correcta de un modelo, de modo que no se ajuste en exceso o en defecto.
- La regularización es una técnica muy importante para reducir el sobreajuste en los modelos de aprendizaje automático. Consiste en añadir una medida de complejidad (término de regularización) a la función de error durante el proceso de entrenamiento.
- Las normas L1 y L2 son las dos medidas de complejidad más utilizadas en la regularización.
- El uso de la norma L1 conduce a la regularización L1, o regresión lasso. El uso de la norma L2 conduce a la regularización L2, o regresión ridge.
- La regularización L1 se recomienda cuando nuestro conjunto de datos tiene numerosas características, y queremos convertir muchas de ellas en cero. La regularización L2 se recomienda cuando nuestro conjunto de datos tiene pocas características, y queremos hacerlas pequeñas pero no nulas.